Touching and Feeling the Data: A Reusable Software Pipeline for Tactile Statistical Graphs in Accessible Education

Lawrence Obiuwevwi, Krzysztof J. Rechowicz, Jessica M. Johnson, Erika Frydenlund,
Vikas Ashok, Sachin Shetty, & Sampath Jayarathna
Old Dominion University, Norfolk, VA, USA
lobiu001@odu.edu, krechowi@odu.edu, j17johnso@odu.edu, efrydenl@odu.edu,
vganjiqu@cs.odu.edu, sshetty@odu.edu, sampath@cs.odu.edu

***Abstract*—Statistical visualization is usually treated as a visual medium, but data can also be touched. Three dimensional printed tactile graphs let blind and low vision students feel distributions, trace trends, and explore relationships through direct haptic interaction. Yet classroom scale use remains limited because producing each graph in CAD software requires specialized skill and hours of manual work. We address this bottleneck as a software problem through a three layer reusable pipeline in about 1500 lines of JavaScript. The first layer derives tactile design parameters automatically from plate dimensions using tactile perception research. The second provides shared chart scaffolding and five modular builders for scatter, bar, histogram, line, and box plots. The optional third layer uses a multi-modal large language model to extract structured chart specifications from uploaded images, with mandatory teacher review before print generation. The pipeline produces print ready binary Standard Tessellation Language files in under 250 milliseconds. We present the design, performance, and limitations.**

***Index Terms*—tactile graphics, 3D printing, accessible education, haptic data exploration, software reuse, LLM-assisted extraction, visually impaired education.**

Fig. 1. Students using 3D-printed tactile learning materials during a collaborative accessibility-focused activity. The reusable raised-surface plates support hands-on exploration of braille and tactile graphical representations in an inclusive learning environment.

I. Introduction

For sighted students, a histogram or scatter plot communicates shape, trend, and outliers almost immediately. For blind or low-vision students [1], [2], the same chart often becomes a verbal description delivered by a screen reader [3]–[5], shifting graphical information into sequential prose and removing the spatial and relational properties that make charts effective [6]–[9]. Tactile graphics address this gap by turning charts into raised physical forms that can be explored through touch [10]. FDM-printed charts encode axes, data features, and Braille at distinct heights, creating a richer tactile hierarchy than embossed paper [11], and consumer FDM printers now cost effective abd affordable.

Despite this, we are aware of no systematic classroom deployment of per-lesson 3D-printed statistical graphs. In our engagement with one statistics course (institution withheld for review), the limiting factor was not printer access but software: manually modeling each chart in Autodesk Fusion 360 required about two hours per graph. Existing commercial tools such as TactileView and ViewPlus IVEO target swell-paper rather than 3D printing and expose no reusable parameter logic [12], [13]. Research systems are typically one-off artifacts [14], [15].

We present a three-layer reusable pipeline that produces a print-ready binary STL in under one second from plate dimensions, chart type, and numeric data. Contributions are: *(i)* a parameter derivation layer grounded in tactile perception research [10], [16]; *(ii)* a modular geometry layer with five chart builders sharing a common scaffolding; and *(iii)* an optional vision-assisted extraction layer converting chart images into editable specifications via a multimodal LLM. To our knowledge this is the first open-source pipeline that automatically generates 3D-printed tactile statistical graphs from either typed data or chart images, combining research-grounded parameter derivation with single-pass Braille-plus-English labeling.

II. Related Work

A. Tactile graphics and 3D-printed accessibility

BANA guidelines [11] and Library of Congress specifications [16] define minimum feature heights, line widths, and

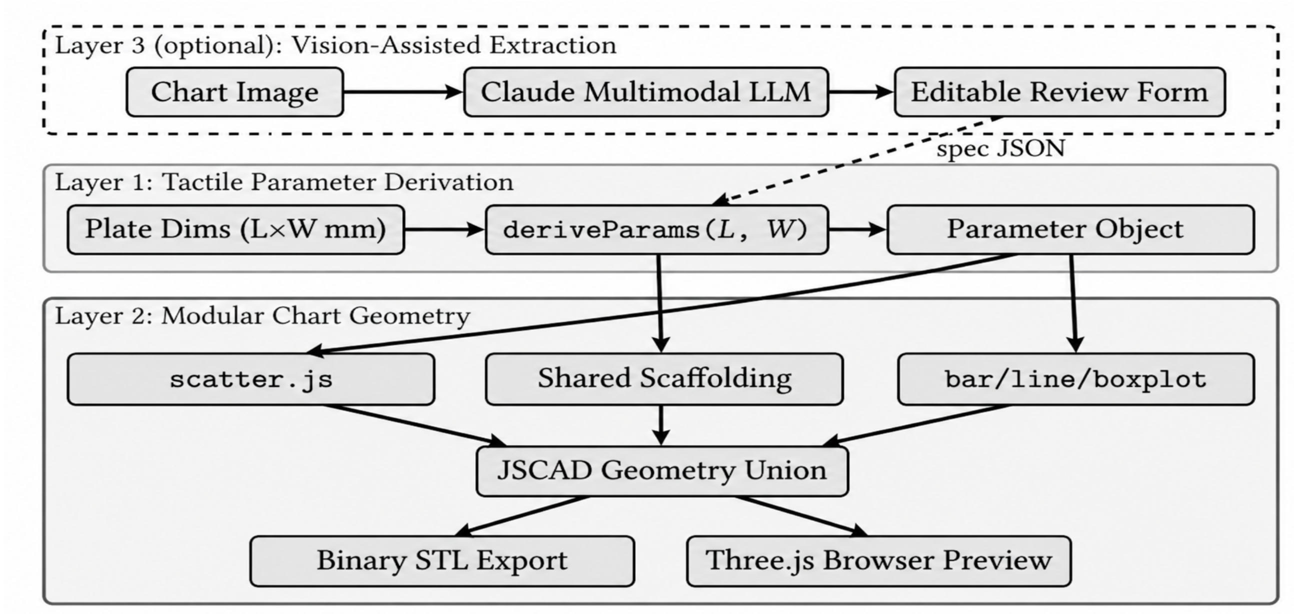

Fig. 2. Three-layer reusable pipeline architecture. Layer 3 (dashed, yellow) is optional; the core pipeline comprises Layer 1 (tactile parameter derivation, blue) and Layer 2 (modular chart geometry, green). Data flows from plate dimensions and an optional chart image through to a binary STL file and a Three.js browser preview.

Braille dot geometry for fingertip reading; Lederman and Klatzky established the 0.5 mm height discriminability floor that anchors our parameter derivation [10], and Weinstein's two-point discrimination findings [17] motivate the 3.5 mm scatter-point spacing floor. Kane et al. showed FDM printing can produce usable tactile overlays for accessibility [14]; Shi et al. confirmed that properly spaced raised features significantly outperform verbal descriptions in comprehension [15]; and Hurst and Kane examined barriers to sharing accessible maker artifacts [18]. Zhao et al. highlight the specific need for automated accessible STEM materials in the classroom [19]. A common theme is that design logic is embedded in individual artifacts rather than exposed as reusable software.

B. Chart understanding and accessible visualization

Academic research on automated tactile chart generation has largely followed the swell-paper embosser paradigm, focusing on transcription quality rather than software architecture [20]. On the vision side, Luo et al.'s ChartOCR [21] and Masry et al.'s ChartQA benchmark [22] demonstrated data extraction and question answering over chart images; Han et al. showed instruction-tuned vision models can treat extraction as a prompt-based task [23], and Lee et al.'s VisText advances semantically rich chart captioning for downstream accessibility [24]. Broader accessibility surveys confirm that multiple modalities are needed to serve learners with diverse visual needs [25].

III. System Design

A. Tactile Parameter Derivation

Every chart generation begins with `deriveParams(L, W)`, which takes plate length L and width W in millimeters and returns a parameter object used by all downstream modules. Plate dimensions are clamped to $[80, 250]$ mm. Baseplate thickness scales mildly with the smaller dimension:

$$t_b = \mathrm{clamp}\left(\frac{\min(L, W)}{60},\ 2.0,\ 3.5\right)\ \mathrm{mm} \qquad (1)$$

Plot margins ensure a fixed 18 mm floor to accommodate dual-format labels (printed English plus one Braille line), with a plate-proportional fallback of $0.14\times$ the relevant dimension.

Feature heights are derived from the 0.5 mm discriminability floor of Lederman and Klatzky [10]: data features 1.5 mm ($3\times$ minimum), axis rails 2.5 mm ($\approx$1.67$\times$ data features), scatter/bar/box heights 1.8 mm, line stroke 1.5 mm, and Braille dots 0.6 mm (Library of Congress standard [16]: 1.5 mm diameter, 2.5 mm intra-cell spacing, 6.0 mm inter-cell spacing). Axis rails are taller than data features so a sweeping finger distinguishes structural boundaries from encoded data. Scatter point minimum separation is 3.5 mm, exceeding the 2–3 mm two-point discrimination threshold [17].

B. Modular Chart Geometry

All geometry is generated with JSCAD (`@jscad/modeling`), running fully in memory and exporting STL-ready meshes with no graphical environment.

Shared scaffolding: The `baseGeometry` module creates the rectangular baseplate, axis rails, and tick marks (1.0 mm wide, 2.5 mm outward, 1.8 mm tall). Every chart module calls these three builders before adding chart-specific geometry.

Chart modules: (i) *Scatter*: point cylinders ($r = 1.6$ mm, $h = 1.8$ mm); colliding points within 3.5 mm are merged. (ii) *Bar*: width is plot width divided by category count (minimum 4.0 mm); negative values supported. (iii) *Histogram*:

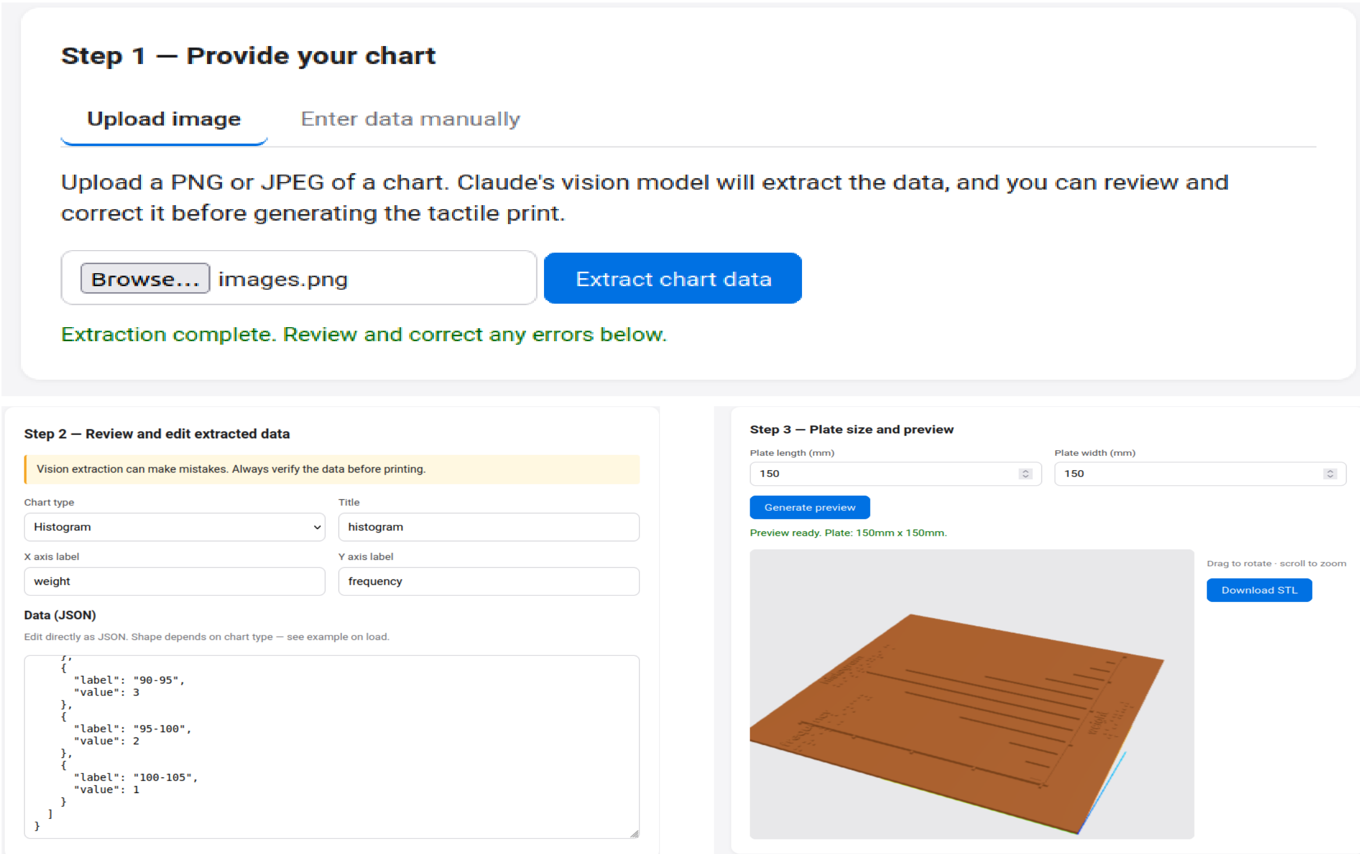

Fig. 3. Top: teacher workflow - chart image upload, Claude vision extraction, and editable review form. Bottom left: extracted data in the review form. Bottom right: Three.js preview of a 150×150 mm tactile histogram plate with raised bars, axis rails, and dual-format labels.

bin count set by Sturges' rule [26], clamped to $[5, 12]$, then rendered through the bar pipeline. (iv) *Line*: rotated cuboid segments (2.0 mm wide, 1.5 mm tall) with vertex cylinders for haptic landmark detection. (v) *Box plot*: IQR box from four cuboid walls; median bar $1.5\times$ IQR wall height; whiskers as 1.2 mm rail with end caps; outliers as scatter-point cylinders at 70% radius.

Dual-format labels: Axis labels and title are printed as JSCAD stroke English text (6 mm glyph height, 0.8 mm raised) above Grade 1 Braille within the reserved 18 mm margin. Capital and number indicators are supported; the vision prompt returns lowercase labels unless capitalization is semantically necessary, reducing cell count.

C. Vision-Assisted Extraction and Implementation

The optional Layer 3 accepts PNG, JPEG, or WebP chart images and returns a structured JSON chart specification via a multimodal LLM (Anthropic Claude). The prompt identifies chart type and outputs typed data: $\{x, y\}$ arrays for scatter and line charts, label–value pairs for bar and histograms, and five-number summaries for box plots. Review is mandatory before STL generation.

The server runs on Node.js (v18+) with Express. Geometry is serialized to binary STL via `@jscad/stl-serializer`; the frontend provides a Three.js preview through a native ES module import map with no build step. The codebase is about 1,500 lines across 15 files. The Anthropic API key is required only for image extraction; the geometry pipeline runs entirely locally.

IV. Results

A. Pipeline Performance

Table I reports STL generation results from `test-pipeline.js` on a 150×150 mm plate. All five chart types complete in under 60 ms, well within the 250 ms threshold perceived as instantaneous [27]. Scatter and box plots have higher primitive counts due to cylinder tessellation; cuboid-heavy types are faster and smaller.

TABLE I
Pipeline Performance by Chart Type (150×150 mm plate)

Chart type	Primitives	STL (KB)	Triangles	Time (ms)
Scatter	214	246.6	5,048	51
Bar	191	170.0	3,480	35
Histogram	172	162.4	3,324	26
Line	172	181.7	3,720	25
Box plot	218	199.9	4,092	24

All five output files satisfy the binary STL formula ($84 + 50T$ bytes exactly), confirming well-formed output with no truncation. Files open directly in PrusaSlicer, Bambu Studio, and Cura with no repair required; geometry is flat-on-bed with no overhangs.

B. Workflow and Extraction Assessment

The baseline workflow required approximately two hours of Fusion 360 modeling per chart. The pipeline reduces this to under 60 ms on-device (excluding vision API round-trip), a reduction of approximately four orders of magnitude in software-side labor.

Vision extraction over an informal sample of textbook chart images correctly identified chart type in all cases and recovered most numeric values within ± 5–10% visual estimation error. Two failure modes were observed: (i) title-case labels generating unnecessary capital indicators in Grade 1 Braille, causing margin overflow , resolved by instructing the model to return lowercase labels unless semantically necessary; and (ii) excessive numeric precision (e.g., 12.3456 for a visually read value of 12), correctable via the mandatory editable review step before STL generation.

V. Discussion and Conclusion

The layered architecture enables reuse at multiple levels. A researcher can swap the parameter derivation layer and test its effect across all five chart types. A developer adding a new chart type need only implement one module; scaffolding, labels, and STL export are reused automatically. The vision extraction layer reflects a broader principle: multimodal AI is most useful here as a *data-entry accelerator*, not as an autonomous producer of accessible artifacts, making mandatory editable review a safety requirement. The core finding is that tactile statistical graphics can be produced fast enough and simply enough for routine classroom use. Future work should conduct formal user studies measuring student comprehension and teacher task-completion time; add Grade 2 Braille support to reduce label cell counts; integrate with matplotlib and ggplot figure objects to remove manual data entry; and expand the supported chart set to violin plots, heat maps, and cumulative distribution functions.

Acknowledgement

This work is supported in part by NSF 245523. Any opinions, findings, and conclusions or recommendations expressed in this material are the author(s) and do not necessarily reflect those of the sponsors.

References

[1] Y. Prakash, A. Kolgar Nayak, S. Jayarathna, H.-N. Lee, and V. Ashok, "Understanding low vision graphical perception of bar charts," in *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*, 2024, pp. 1–10.

[2] Y. Prakash, P. A. Khan, A. K. Nayak, S. Jayarathna, H.-N. Lee, and V. Ashok, "Towards enhancing low vision usability of data charts on smartphones," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 1, pp. 853–863, 2024.

[3] Y. Prakash, A. Kolgar Nayak, M. Shoaib Alyaan, S. Jayarathna, H.-N. Lee, and V. Ashok, "VoxVista: Enhancing screen reading experience for online user comments," in *Proceedings of the 2026 Conference on Human Information Interaction and Retrieval*, 2026, pp. 285–294.

[4] M. Sunkara, A. Kolgar Nayak, S. Kalari, Y. Prakash, S. Jayarathna, H.-N. Lee, and V. Ashok, "Quickque: Enabling quick access to information in user reviews for screen reader users," in *Proceedings of the 22nd International Web for All Conference*, 2025, pp. 22–24.

[5] M. J. Ferdous, A. Kolgar Nayak, Y. Prakash, N. Venkatraman, S. Jayarathna, H.-N. Lee, and V. Ashok, "Understanding online discussion experiences of blind screen reader users," *International Journal of Human–Computer Interaction*, pp. 1–31, 2025.

[6] E. R. Tufte, *The Visual Display of Quantitative Information*. Cheshire, CT: Graphics Press, 1983.

[7] A. Lundgard and A. Satyanarayan, "Accessible visualization via natural language descriptions: A four-level model of semantic content," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 1073–1083, 2022.

[8] N. W. Kim, S. C. Joyner, A. Riegelhuth, and Y. Kim, "Accessible visualization: Design space, opportunities, and challenges," *Computer Graphics Forum*, vol. 40, no. 3, pp. 173–188, 2021.

[9] K. Marriott, M. Butler, M. Duckham, T. Dwyer, C. Goncu, J. Gulliksen, L. Holloway, S. K. Kane, S. Kurniawan, M. R. Morris, and H. Petrie, "Inclusive data visualization for people with disabilities: A call to action," *ACM Interactions*, vol. 28, no. 3, pp. 47–51, 2021.

[10] S. J. Lederman and R. L. Klatzky, "Hand movements: A window into haptic object recognition," *Cognitive Psychology*, vol. 19, no. 3, pp. 342–368, 1987.

[11] Braille Authority of North America (BANA), *Guidelines and Standards for Tactile Graphics*, 2016. [Online]. Available: https://www.brailleauthority.org/braille-formats-principles-print-braille-transcription-2016

[12] Humanware, "TactileView software," [Online]. Available: https://store.humanware.com/hus/blindness, accessed 2025.

[13] ViewPlus Technologies, "IVEO hands-on learning system," [Online]. Available: https://viewplus.com, accessed 2025.

[14] S. K. Kane, J. P. Bigham, and J. O. Wobbrock, "Slide rule: Making mobile touch screens accessible to blind users using multi-touch interaction techniques," in *Proc. ACM ASSETS*, 2008, pp. 73–80.

[15] L. Shi, Y. Zhao, and S. Azenkot, "Designing interactions for 3d printed models with blind people," in *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility*, 2017, pp. 200–209.

[16] Library of Congress, National Library Service for the Blind and Print Disabled, *Braille Formats: Principles of Print-to-Braille Transcription*, 2016. [Online]. Available: https://www.loc.gov/nls/braille-audio-reading-materials/braille/

[17] S. Weinstein, "Intensive and extensive aspects of tactile sensitivity as a function of body part, sex, and laterality," in *The Skin Senses*, D. R. Kenshalo, Ed. Springfield, IL: Charles C. Thomas, 1968, pp. 195–222. [Online]. Available: https://api.semanticscholar.org/CorpusID:56508611

[18] A. Hurst and S. Kane, "Making 'Making' accessible," in *Proc. ACM IDC*, 2013, pp. 635–638.

[19] H. Palani, R. Adnin, and S. Nagar, "Kanak: Automating the generation of accessible STEM materials for blind and low-vision students," in *Proceedings of the 37th Australian Conference on Human-Computer Interaction*, 2025, pp. 287–299.

[20] J. Rowell and S. Ungar, "The world of touch: An international survey of tactile maps. Part 1: Production," *British Journal of Visual Impairment*, vol. 21, no. 2, pp. 98–104, 2003.

[21] J. Luo, C. Li, J. Yang, and C. Zhang, "ChartOCR: Data extraction from charts images via a deep hybrid framework," in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2021, pp. 1916–1924.

[22] A. Masry, D. Long, J. Q. Tan, S. Joty, and E. Hoque, "ChartQA: A benchmark for question answering about charts with visual and logical reasoning," in *Findings of the Association for Computational Linguistics: ACL 2022*, 2022, pp. 2263–2279.

[23] Y. Han, C. Zhang, X. Chen, M. Cui, X. Yang, Z. Wang, and C. Zhang, "ChartLlama: A multimodal LLM for chart understanding and generation," arXiv preprint arXiv:2312.05729, 2023.

[24] B. Tang, A. Boggust, and A. Satyanarayan, "Vistext: A benchmark for semantically rich chart captioning," in *Proc. ACL*, 2023, pp. 7268–7298.

[25] C. Jung, S. Mehta, A. Kulkarni, Y. Zhao, and Y.-S. Kim, "Communicating visualizations without visuals: Investigation of visualization alternative text for people with visual impairments," in *Proc. ACM CHI*, 2022.

[26] H. A. Sturges, "The choice of a class interval," *J. American Statistical Association*, vol. 21, no. 153, pp. 65–66, 1926.

[27] R. B. Miller, "Response time in man-computer conversational transactions," in *Proc. AFIPS Fall Joint Computer Conf.*, vol. 33, 1968, pp. 267–277.